

Day 10: Autoregressive Inference

- **Autoregressive Inference** — Decoding strategies and KV cache
- 3 hours lecture + practical
- Slides, notes, and code on the course site

- Text Generation Loop
- KV Cache
- Efficient Attention
- Serving & Systems

Text Generation Loop



Autoregressive Decoding

- Start from prompt tokens
- Repeat: forward pass $\rightarrow$ logits $\rightarrow$ sample/argmax $\rightarrow$ append
- Stop at EOS or max length

WELCOME TO THE

UCL x DeepMind lecture series

In this lecture series, leading research scientists from leading AI research lab, DeepMind, will give 12 lectures on an exciting selection of topics in Deep Learning, ranging from the fundamentals of training neural networks via advanced ideas around memory, attention, and generative modelling to the important topic of responsible innovation.

Please join us for a deep dive lecture series into Deep Learning!

#UCLxDeepMind



Text Generation Loop — Autoregressive Decoding (source: course materials)

Greedy & Beam Search

- Greedy: $x_t = \arg \max p(x_t | x_{<t})$
- Beam search keeps top- k partial sequences
- Deterministic but often dull

TODAY'S LECTURE

Modern Latent Variable Models and Variational Inference

Latent variable models provide a powerful and flexible framework for generative modelling. After introducing this framework along with the concept of inference, which is central to it, this lecture will focus on two types of modern latent variable models: invertible models and intractable models. Special emphasis will be placed on understanding variational inference as a key to training intractable latent variable models.



Text Generation Loop — Greedy & Beam Search (source: course materials)

- Temperature τ : soften $\text{softmax}\left(\frac{\text{logits}}{\tau}\right)$
- Top- k and nucleus (top- p) filtering
- Repetition penalty

Lecture Outline

1

Generative
Modelling

2

Latent Variable
Models &
Inference

3

Invertible Models &
Exact Inference

4

Variational
Inference

5

Gradient
Estimation in VI

6

Variational
Autoencoders



Text Generation Loop — Sampling Methods (source: course materials)

Latency Metrics

- Time to first token (TTFT)
- Tokens per second (TPS)
- Prefill vs decode phases

DeepMind

1

Generative Modelling



Text Generation Loop — Latency Metrics (source: course materials)

KV Cache



- Self-attention recomputes keys/values for all past tokens
- At step t , past K, V are unchanged
- Cache avoids $O(t^2)$ redundant work per step

What are generative models?

- Generative models are probabilistic models of high-dimensional data
 - Describe the probabilistic process of generating an observation
 - The emphasis is on capturing the dependence between the dimensions
 - Provide a way of generating new datapoints
- Historically, generative modelling was considered to be a subfield of unsupervised learning (i.e. learning with no labels).
 - Conditional generative models make the boundary between supervised and unsupervised learning blurry.
- Generative models can be used for essentially any kind of data:
 - Text, images, audio, biological sequences, etc.



KV Cache — Motivation (source: course materials)

- Store K, V per layer per head
- Memory $O(L \cdot H \cdot T \cdot d_h)$ grows with context
- Batching pads to max length in batch

Progress in generative models

Samples from 2014:



Deep AutoRegressive Networks, Gregor et al.



KV Cache — Cache Structure (source: course materials)

Prefill vs Decode

- Prefill: process prompt in parallel (compute-bound)
- Decode: one token at a time (memory-bandwidth)
- Continuous batching in serving systems

Progress in generative models

Samples from 2019:



Hierarchical Autoregressive Image Models with Auxiliary Decoders, De Fauw et al.



KV Cache — Prefill vs Decode (source: course materials)

- Share K,V heads across query heads
- Reduces cache size with minimal quality loss
- Standard in modern LLM inference

Autoregressive models

- Model the 1-dimensional conditional distributions instead of modelling the joint distribution directly:
 - Based on the chain rule: $p(\mathbf{x}) = \prod_{d=1}^D p(x_d | x_1, \dots, x_{d-1})$
 - Trained with maximum likelihood
- Pros
 - 1-dimensional distributions are easy to model
 - Simple and efficient training
 - No sampling at training time
- Cons
 - Slow, sequential generation — one dimension at a time
 - Usually much better at modelling local structure than global structure



KV Cache — Multi-Query / GQA (source: course materials)

Efficient Attention



- Tiled softmax without materializing full $n \times n$
- IO-aware — faster on GPU memory hierarchy
- Training and prefill benefit most

Generative Adversarial Networks

- Model: A neural net that maps noise vectors to observations
- Training: use the learning signal from a classifier trained to discriminate between samples from the model and the training data
- Pros
 - Can generate very realistic images
 - Conceptually simple implementation
 - Fast generation
- Cons
 - Cannot be used to compute probability of observations
 - "Mode collapse": Models ignore regions of the data distribution
 - Training can be unstable and requires many tricks to work well

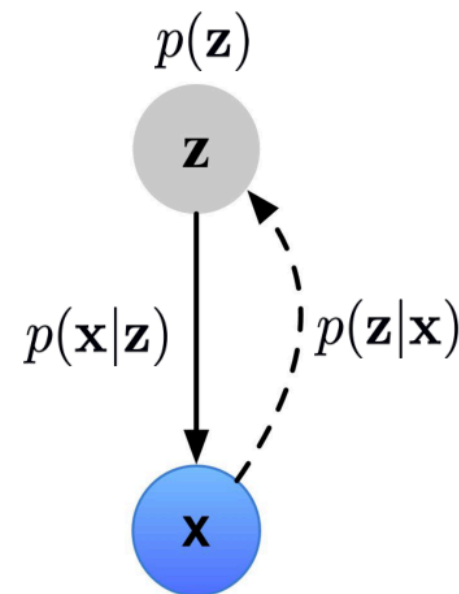


Efficient Attention — FlashAttention (source: course materials)

- Non-contiguous KV blocks like virtual memory
- Reduces fragmentation in batched serving
- Higher GPU utilization

Latent variable models

- A latent variable model (LVM) defines a distribution over observations $\mathbf{x}$ by using a (vector) latent variable $\mathbf{z}$ and specifying:
 - The **prior** distribution $p(\mathbf{z})$ for the latent variable
 - The **likelihood** $p(\mathbf{x}|\mathbf{z})$ that connects the latent variable to the observation
- The prior and the likelihood define the **joint** distribution $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$
- We will be interested in computing the **marginal likelihood** $p(\mathbf{x})$ and the **posterior** distribution $p(\mathbf{z}|\mathbf{x})$.



Efficient Attention — PagedAttention (vLLM) (source: course materials)

Speculative Decoding

- Draft model proposes several tokens
- Target model verifies in parallel
- Acceptance rate determines speedup

Inference

- **Inference** refers to computing the posterior distribution for the given observation:

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})} = \frac{p(\mathbf{x}, \mathbf{z})}{\int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}}$$

- This requires solving the important sub-problem of computing the **marginal likelihood** of the observation:

$$p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$



Efficient Attention — Speculative Decoding (source: course materials)

- RoPE scaling, YaRN, ALiBi
- Ring attention for very long sequences
- Context window vs true reasoning

Lecture 04

Latent Spaces, Neural network architectures

MIT IAP 2026 | Jan 28, 2025

Peter Holderrieth and Ron Shprints

Sponsor: Tommi Jaakkola



Efficient Attention — Long Context (source: course materials)

Serving & Systems

- Weight-only INT4 (GPTQ, AWQ)
- KV cache quantization
- Accuracy vs throughput tradeoffs

Batching Strategies

- Static vs continuous batching
- Request scheduling and preemption
- Multi-tenant SLA constraints

- Week 1: ML/DL foundations → Transformers
- Week 2: generative models → production LLM inference
- Final assessment ties math + code together

- Day 10: **Autoregressive Inference**
- Decoding strategies and KV cache
- Questions welcome — practical follows

Questions?

Practical session follows — see course site.